

* Arithmetic Instructions

1) Normal Addition

ADD r_0, r_1, r_2 ; $(r_0) \leftarrow (r_1) + (r_2)$; Flag will remain unchanged
Flag register will not update.

2) ~~add~~ r_0, r_1, r_2 ; $(r_0) \leftarrow (r_1) + (r_2)$; Flag register will be updated.

3) ~~addes~~ r_0, r_1, r_2 ; Conditional addition with flag update
It will perform addition only if zero flag is equal to 1 ($Z=1$) and after addition operation it will update status flag.

4) ~~adc~~ r_0, r_1, r_2 ; Addition with carry and status flag will not change
 $(r_0) \leftarrow (r_1) + (r_2) + \text{Carry}$

5) ~~adcs~~ r_0, r_1, r_2 ; Addition with carry and status flag will be updated.

6) ~~addcqs~~ r_0, r_1, r_2 ; Conditional addition with carry and status flag will be updated.

7) ~~Sub~~ r_0, r_1, r_2 ; $(r_0) \leftarrow (r_1) - (r_2)$; Normal Subtraction

8) ~~sbc~~ r_0, r_1, r_2 ; $(r_0) \leftarrow (r_1) - (r_2) - \text{Borrow}$

* Reverse Subtraction:

9) ~~rsb~~ r_0, r_1, r_2 ; $(r_0) \leftarrow (r_2) - (r_1)$

~~rsc~~ r_0, r_1, r_2 ; $(r_0) \leftarrow (r_2) - (r_1) - \text{borrow}$

Example: ~~cmp~~ r_0, r_1
~~addne~~ r_6, r_5, r_4 ; $(r_6) \leftarrow (r_5) + (r_4)$ only if $ZF=0$ or $(r_0) \neq (r_1)$

* Examples for Syntax of Conditional Instructions:

CS $\rightarrow$ Carry set; $C=1$

CC $\rightarrow$ carry clear/zero; $C=0$

VS $\rightarrow$ Overflow set, $V=1$

VC $\rightarrow$ Overflow clear, $V=0$

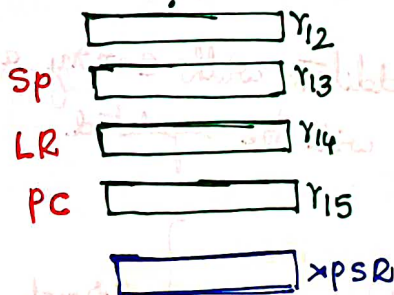
EQ $\rightarrow$ Equal, $Z=1$

NE $\rightarrow$ Not equal, $Z=0$

MI $\rightarrow$ Minus, $N=1$

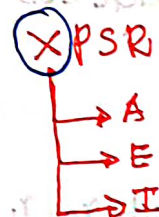
PL $\rightarrow$ plus, $N=0$

* Basics:



* The program status register is composed of three status registers

- (1) Application PSR
- (2) Execution PSR
- (3) Interrupt PSR



	31	30	29	28	27	0
APSR	N	Z	C	V	Q	
IPSR						
EPSR						

NOTE:

SP $\rightarrow$ Stack pointer

LR $\rightarrow$ Link Register

PC $\rightarrow$ Program counter

PSR $\rightarrow$ program status Register

NOTE:

- 1) `cmp` - compare $\rightarrow$ Flags set to result of subtraction
- 2) `cmn` - compare Negative $\rightarrow$ Flags set to result of Addition

* Comparison of `adds` and `cmn` Instructions:

1) `ldr r0, #0xffffffff`

`adds r0, #1`

Before execution, `0xffffffff` `r0`

After execution, `0x00000000` `r0`

$N=0; Z=1; C=1; V=0$

2) `ldr r0, #0xffffffff`

`cmn r0, #1`

Before execution, `0xffffffff` `r0`

After execution, `0xffffffff` `r0`

$N=0; Z=1; C=1; V=0$

* Comparison of `subs` and `cmp` Instructions:

1) `mov r0, #0x27`

`mov r1, #0x20`

`subs r0, r1`

Before Execution

`r0` `0x00000027`

`r1` `0x00000020`

After Execution

`r0` `0x00000007`

`r1` `0x00000020`

$N=0; Z=0; C=1; V=0$

2) `mov r0, #0x27`

`mov r1, #0x20`

`cmp r0, r1`

Before Execution

`r0` `0x00000027`

`r1` `0x00000020`

After Execution

`r0` `0x00000027`

`r1` `0x00000020`

$N=0; Z=0; C=1; V=0$

`27 = 0100 0111`

`20 = 0100 0000`

is

`1011 1111`

1

`11 000000`

`0100 0111`

`cy = 1` `0 000 0111`

0 7

* Compare Instruction: It performs $r_1 - r_2$ but does not store result of $\text{Cmp } r_1, r_2$; if $r_1 > r_2$ then $\text{Carry} = 1$ it.
if $r_1 < r_2$ then $\text{Carry} = 0$
if $r_1 = r_2$ then $\text{Zero} = 1$

NOTE: It is just affecting flag for comparison, here 's' is not required to change the status.

* Logical Instructions:

1) Normal AND Operation: (Bit wise manipulation)

$\text{AND } r_0, r_1, r_2$; $(r_0) \leftarrow (r_1) \text{ AND } (r_2)$; it will not update the flags

2) $\text{ands } r_0, r_1, r_2$; $(r_0) \leftarrow (r_1) \text{ AND } (r_2)$; it will update the flags

3) $\text{orr } r_0, r_1, r_2$; $(r_0) \leftarrow (r_1) \text{ OR } (r_2)$; OR operation

4) $\text{orrs } r_0, r_1, r_2$; $(r_0) \leftarrow (r_1) \text{ OR } (r_2)$; it will update the flags

5) EOR - Logical XOR operation

$\text{eor } r_0, r_1, r_2$; $(r_0) \leftarrow (r_1) \text{ XOR } (r_2)$

$\text{eorrs } r_0, r_1, r_2$; $(r_0) \leftarrow (r_1) \text{ XOR } (r_2)$; it will update the flags.

NOTE: The above logical Instructions supports Conditional execution.

Example: $\text{andeqs } r_0, r_1, r_2$; performs logical AND operation only if $Z=1$ and after performing operation it will update the flags.

$\text{orrccs } r_0, r_1, r_2$; performs logical OR operation only if $C=1$ and after performing operation it will update the flags.

6) BIC - And Operation with complement

$\text{BIC } r_0, r_1, r_2$; $(r_0) \leftarrow (r_1) \text{ AND } (\overline{r_2})$

7) TST - Test logic AND operation

TST R0, R1 ; R0 AND R1
 ; It performs R0 AND R1, but does not store result of it.
 ; It updates the flags

NOTE :

Cmp, Cmn & TST } → These Instructions affects the flags and does not stores the result of operation.
 TEQ
 Here, 'S' is not required to change the status.

8) TEQ - Test logic XOR operation

TEQ R0, R1 ; R0 XOR R1
 ; It performs R0 XOR R1, but does not stores result of it.
 ; It updates the flags.

* Branch Instructions :

↳ changes the flow of execution or is used to call a routine

↳ types of Branch Instructions:

↳ Branch : B

↳ Branch with link : BL

↳ Branch exchange : Bx

↳ Branch exchange with link : BLx

* B - branch:

↳ B {<cond>} label

Example: here b here ; unconditional branch Instruction

Examples of conditional branch Instructions :-

↳ bne ; Branch if not equal $Z=0$

↳ beq ; Branch if equal $Z=1$

↳ bcs : Branch if carry $C=1$

↳ bcc : Branch if carry $C=0$

↳ bvs : Branch if overflow, $V=1$

↳ bvc : Branch if overflow, $V=0$

↳ bmi : Branch if minus, $N=1$

↳ bpl : Branch if plus, $N=0$

Some examples of unconditional branch Instructions :-

b forward → Label name
add r1, r2, #4
add r0, r6, #2
add r3, r7, #4
forward sub r1, r2, #4

Label name -----

backward add r1, r2, #4
sub r1, r2, #4

add r4, r6

b backward

↳ Branch : B

↳ Branch with link : BL

↳ Branch exchange : BX

↳ Branch exchange with link : BLX

B-branch:

↳ B {<cond>} label

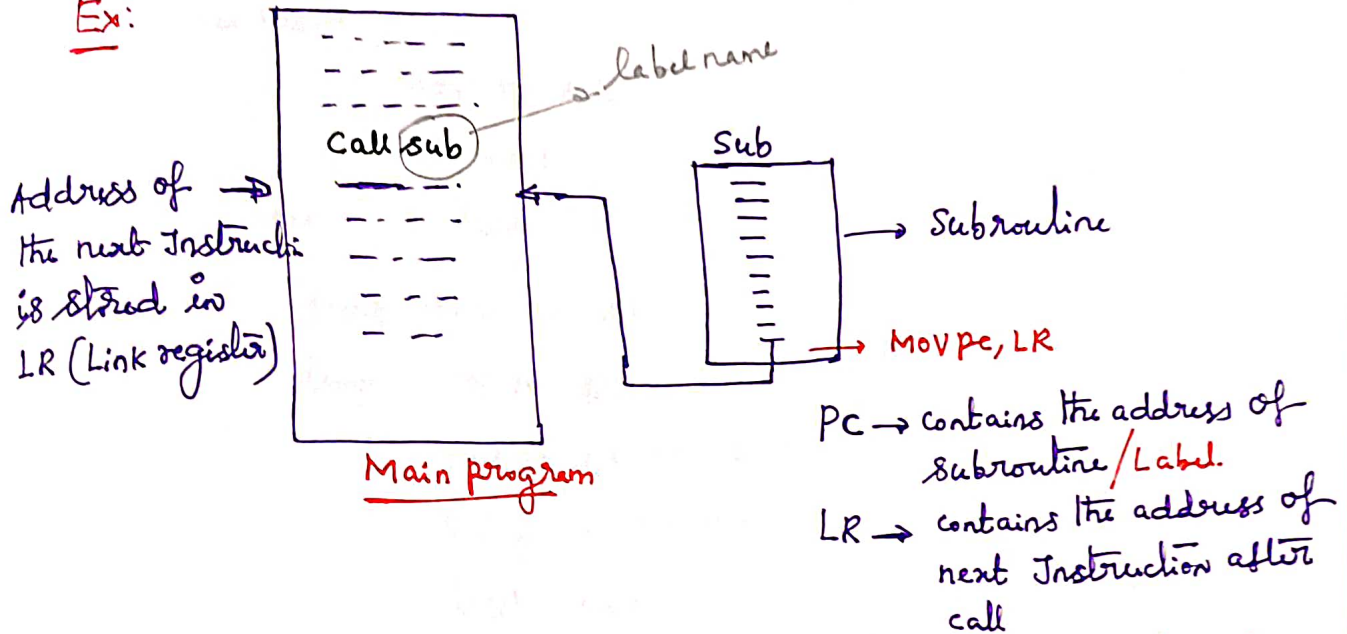
Example: here B is an unconditional branch instruction

BL — branch with link. (call subroutine)

BL {<cond>} label

→ This instruction is used to call subroutine.

Ex:

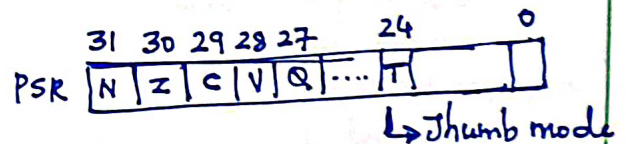


NOTE: At the time of transferring control from the subroutine to main program, the content of LR register is moved to PC.

BLx — branch exchange with link: (call subroutine)

→ It will update Thumb bit ($T=1$)

→ It will invoke Thumb routine



Note: Thumb state: 16-bit Instructions

Bx — branch exchange

→ used to branch to and from Thumb code address.

* ALP to find the factorial of a given number

export —main
area prg3, code, readonly

—main

mov r0, #4

bl fact

here b here

fact mov r3, #1

loop cmp r0, #1

mulgt r3, r0, r3 ;

subgt r0, r0, #1

bgt loop

mov pc, R14

end

$$4 \times 3 \times 2 \times 1$$

$$r_3 = 4 = 4 \times 1$$

$$12 = 3 \times 4$$

$$24 = 2 \times 12 \quad 16 \overline{) 24} \\ 1-8$$

$$24 = 1 \times 24$$

$$(24)_{10} = 18H$$

* Write an ALP to perform 32-bit addition, subtraction, multiplication and division based on conditions.

export —main
area prg3, code, readonly

—main

mov r0, #1 → condition no.

ldr r1, =0xf0000002

ldr r2, =0x00000002

ldr r6, =0x20000020

ldr r7, =0x20000024

Result locations

cmp r0, #1

bne Subb

adds r3, r1, r2

b loop

cmp r0, #2

bne mult

subs r3, r1, r2

b loop

Subb

mult cmp r0, #3
bne ~~loop~~ divi
umull r3, r4, r1, r2
b loop

divi cmp r0, #4
bne loop
udiv r3, r1, r2
b loop

loop str r3, [r6]
str r4, [r7]

here b here
end

Conditions :-

1: Addition

2: Subtraction

3: Multiplication

4: Division

5: No operations (>4)

* Shift and Rotate Instructions:

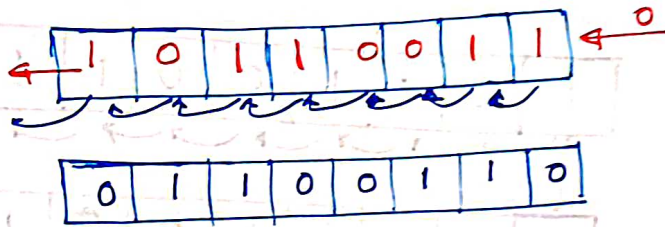
- ↳ LSL: Logical shift left
- ↳ LSR: Logical shift Right
- ↳ ASR: Arithmetic shift Right
- ↳ ASL: Arithmetic shift Left
- ↳ ROR: Rotate Right
- ↳ RRX: Rotate Right extended by 1 bit.

1) LSL: Logical shift left



NOTE: LSB is filled with zero and MSB is discarded

Example:



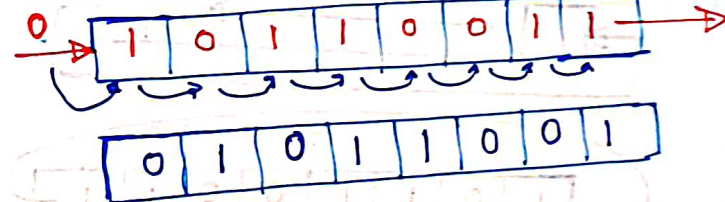
After LSL

2) LSR: Logical shift Right



NOTE: Vacant MSB is filled with zero and LSB is discarded

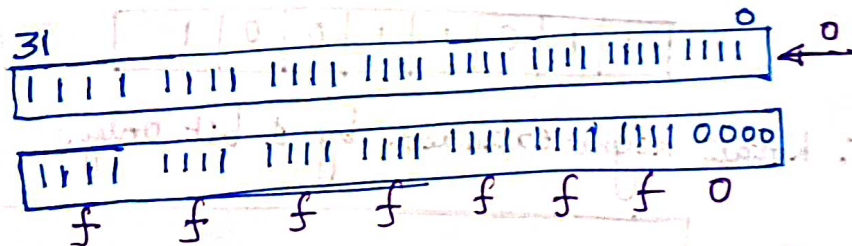
Example:



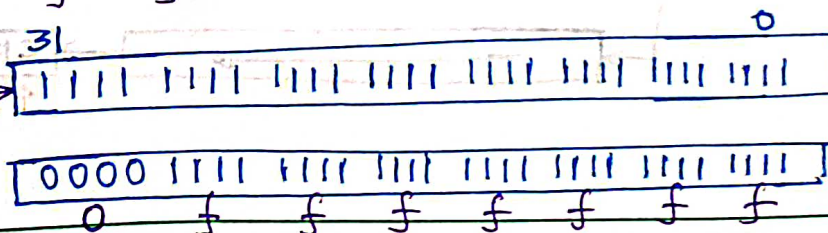
After LSR

Examples

Logical shift left by 4



Logical shift Right by 4 -



NOTE:

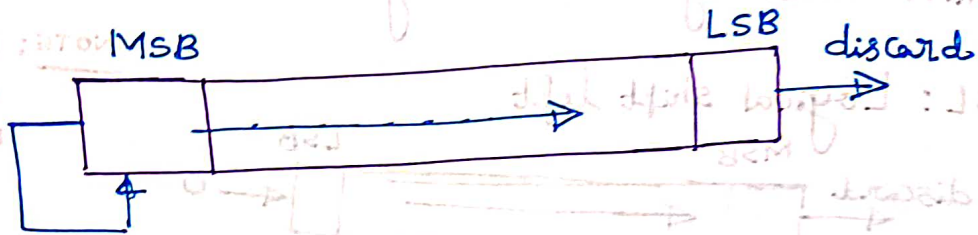
↳ ASL operation is same as LSL operation.

NOTE

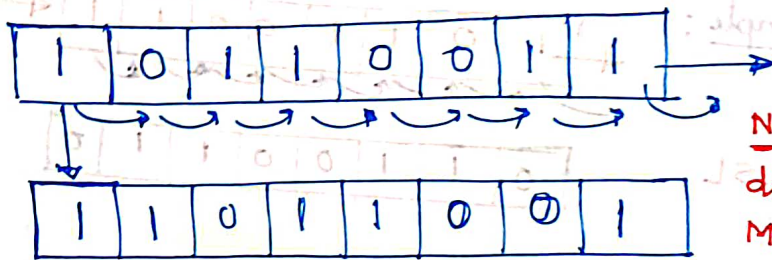
LSL = ASL

3) ASL: Arithmetic shift left

4) ASR: Arithmetic shift Right

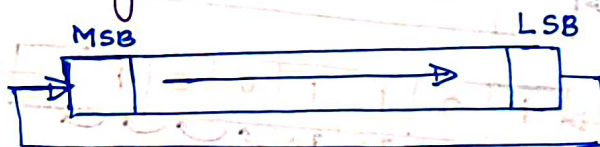


After ASR

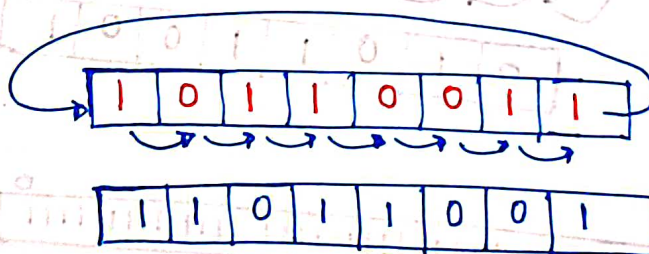


NOTE: LSB is discarded and MSB is filled with previous value.

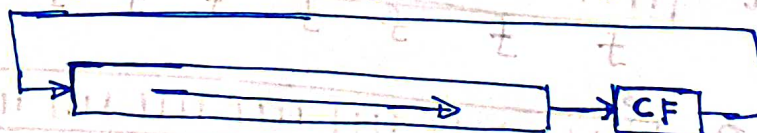
5) ROR: Rotate Right



After ROR



6) RRX: Rotate Right extended by 1 bit only.



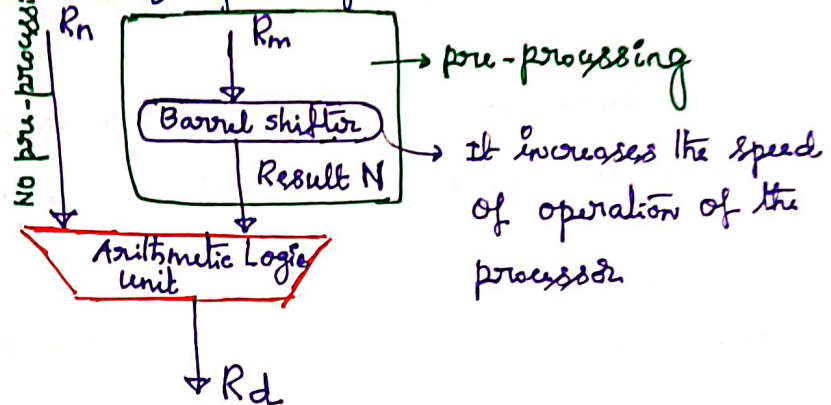
↑ carry flag

NOTE: Do not specify a shift amount for RRx. RRx is always a one-bit shift.

Example: `MOV r4, r4, RRx #3`; This instruction is invalid since RRx is always a one-bit shift.

This instruction performs RRx operation on r4 register by 3 times. The result is stored in r4 register.

* Barrel shifter: The ARM arithmetic logic unit has a 32-bit barrel shifter that is capable of shift and rotate operations.



* Examples:

↳ `mov r0, r1, ASL #3`; 3 times left shift operations will be performed on the content of r1 and the result will be placed into r0.

↳ `mov r2, #3`
↳ `mov r0, r1, ASL r2`; performs left shift operations on the content of (r1) by (r2) times.

↳ `mov r0, r1, ASR #3`;

↳ `mov r0, r1, ASR r2`;

↳ `mov r0, r0, LSL #1`;

Multiply (r0) value by two. (unsigned)

Ex:

0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---

 $\leftarrow 0 = 1 \times 2$

0	0	0	0	0	0	1	0
---	---	---	---	---	---	---	---

 $\rightarrow 2$

↳ `mov r1, r1, LSR #2`; Divide (r1) by four.

* Data conversion operations: [Extending 8-bit/16-bit data to 32-bit data]

↳ The instructions available for handling signed and unsigned extensions of data.

↳ Example: To convert 8-bit value to 32-bit
To convert 16-bit value to 32-bit

↳ $SXTB\ R_d, R_m$; Signed extend byte data into word
 $SXTH\ R_d, R_m$; Signed extend half-word data into word

↳ $UXTB\ R_d, R_m$; unsigned extend byte data into word
 $UXTH\ R_d, R_m$; unsigned extend half-word data into word

Examples:

If $r_0 = 0x55aa8765$

65 = 0110 0101

87 = 1000 0111

1) $SXTB\ r_1, r_0$; $r_0 = 0x00000065$

2) $SXTH\ r_1, r_0$; $r_1 = 0xffff8765$

3) $UXTB\ r_1, r_0$; $r_1 = 0x00000065$

4) $UXTH\ r_1, r_0$; $r_1 = 0x00008765$

NOTE: These Instructions are useful for converting between different data types.

* Reverse ordering Instructions:

Examples:

If $r_0 = 0x12345678$

1) $REV\ r_1, r_0$; $r_1 = 0x78563412$; Reverse bytes in word

2) $REVH\ r_2, r_0$; $r_2 = 0x34127856$; Reverse bytes in each half-word

If $r_0 = 0x33448899$

3) $REVSH\ r_1, r_0$; $r_1 = 0xffff9988$; Reverse bytes in bottom half-word and sign extend the result.

NOTE: These Instructions are useful for converting data between little endian and big endian.

STM32CubeMX

while(1)

{

/* user code */

HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_RESET);

// Delay

HAL_Delay(1000);

// Turn off the LED

HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15, GPIO_PIN_RESET);

// Delay

HAL_Delay(1000);

}

NOTE:-

Green LED — Pin 12

Orange LED — Pin 13

Red LED — Pin 14

Blue LED — Pin 15

Options for Target:

↳ click Target → ARM compiler → V6.19

↳ click Debug → ST-Link Debugger → click settings

Debug Adapter

unit

ST-Link/V2-1

Flash Download

- ☒ program
- ☒ verify
- ☒ Reset and Run

NOTE: After Build, Rebuild, we should click load option.

RCC → Reset and clock controller → Manages system and peripherals clocks and manages the various resets present in the clocks.

Sys → system configuration

MDK - ARM (Microcontroller Development kit) : (V5.32)

↳ is the complete software development Environment for ARM7, ARM9, cortex-M processor based devices.

* STM32CubeMx :

↳ is a graphical tool that allows easy configuration of STM32 microcontrollers and microprocessors

↳ Generation of 'C' Initialization code for the ARM cortex-M cores

↳ STM32CubeMx is a part of the STM32 MCU development ecosystem

↳ It can generate Initialization code for STM32 microcontrollers based on our configuration choices.

* Bit-field processing Instructions:

→ Cortex M3 and M4 processors supports a number of bit field processing operations, these operations are very much useful for control applications.

→ BFC → clear bit field within a register

→ BFI → Insert bit field to a register

→ CLZ → Count leading zero

→ RBIT → Reverse bit order in Register

* Examples:

1) LDR r0 = 0x1234FFFF

BFC r0, #4, #8 ; r0 = 0x1234F00F

→ clear 8 bits from the fourth bit of r0.

2) LDR R0, = 0x12345678

LDR R1, = 0x3355AACC

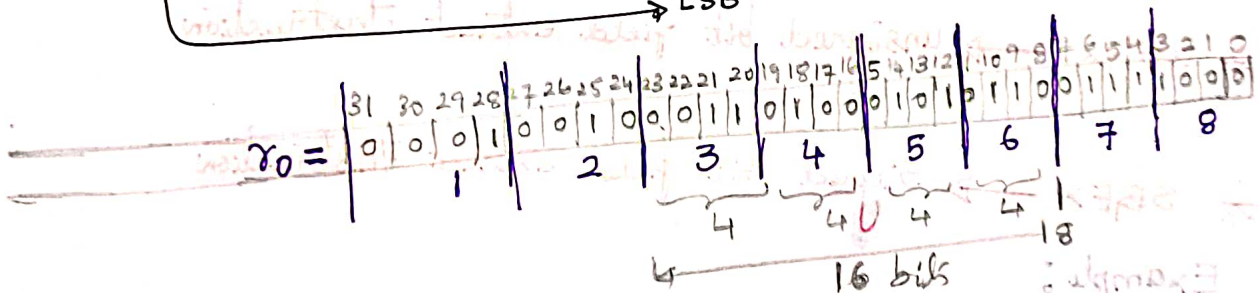
BFI R1, R0, #8, #16

Insert r0[15:0] values into r1[23:8]

∴ r1 = 0x335678CC

Width field

LSB



3) CLZ → Counts the number of leading zeros. It is used in floating point.

4) RBIT → Reverses the bit order in a data word

Example

If r9 = 0xB4E10C23 (1011 0100 1110 0001 0000 1100 0010 0011)

RBIT r0, r1 ; r0 = (1100 0100 0011 0000 1000 0111 0010 1101)
= 0xC430872D

Source Register

Destination Register

NOTE: $REV\ r1, r0$ & $RBIT\ r0, r1$

→ Reverses the bit order in a word
→ Reverses the bytes in a word

Example: Illustration of REV and RBIT Instructions

$LDR\ r1 = 0xB4E10C23$

$RBIT\ r0, r1 ; r0 = 0xC430872D$

$rev\ r0, r1 ; r0 = 0x230CE1B4$

* CLZ → Counts the number of leading zeros.

→ If no bits are set the result is 32

Ex: $\begin{array}{|c|} \hline 31 \\ \hline 000 \dots 0 \\ \hline \end{array} = 32 \text{ zeros}$

→ If all bits are set then the result is zero

Ex: $\begin{array}{|c|} \hline 31 \\ \hline 111 \dots 1 \\ \hline \end{array} = 0$

* $UBFX$ → unsigned bit field extract Instruction.

* $SBFX$ → Signed bit field extract Instruction

Example:

$LDR\ R0 = 0x5678ABCD$

$UBFX\ R1, R0, \#4, \#8 ; R1 = 0x000000BC$ [zero extend of 0xBC]

→ width
→ LSB
BC is extracted

The above instruction extracts a bit field from a register starting from any location (specified by the $\langle \#LSB \rangle$ operand) in this example 4th location with any width (specified by the $\langle \#width \rangle$ operand), in this example width is 8 bits, zero extend it, and puts it in the destination register.

III^{ly}, SBFx extracts a bit field but sign extends it before putting it in a destination register.

Example :

LDR R0, =0x5678ABCD

SBFX R1, R0, #4, #8 ; R1 = 0xFFFFFFFFBC [signed extend of 0xBC]

width
LSB

sign extension